

GRAIL MLX Prototype

Goal-Regressive Adaptive Inference Layer

Mac M5 Max · MLX · Go · Fine-Tuned Adapters

Unified Design Document — Prototype v0.1

Platform	Apple M5 Max · 40-core GPU · 128GB Unified Memory
Runtime	Go (coordination, process management) + Python/MLX (inference sidecars)
Scope	macOS system management: Brew packages, OS updates, zero-day mitigation
Status	<i>Pre-implementation prototype design</i>

Executive Summary

The original GMAIL whitepaper is deliberately implementation-agnostic. Its coordination model — goal regression, gossip-propagated block state, stateless workers, quorum-confirmed dependency discovery — makes no assumptions about inference backend, hardware, or work domain. That agnosticism is architectural: GMAIL describes how coordination works, not what does the work.

This document is its first concrete instantiation. Every variable the whitepaper left open is pinned. The hardware is Apple M5 Max, leveraging unified memory to hold multiple fine-tuned LoRA adapters simultaneously without copy overhead. The inference layer is MLX, optimized natively for Apple Silicon. The work domain is macOS system management — Brew packages, OS updates, zero-day mitigation — chosen because it exemplifies the problem class GMAIL was designed for: operational tasks where the correct sequence of steps cannot be known before work begins.

The whitepaper describes what the system must guarantee. This document describes how those guarantees are built on this machine, for this purpose.

Table of Contents

Executive Summary

1. Architecture Overview

- 1.1 Component Map
- 1.2 Stack Diagram

2. Go ↔ MLX Bridge

- 2.1 MLX Sidecar — Python
- 2.2 Sidecar Pool — Go

3. Worker Execution Model

- 3.1 Deterministic Workers
- 3.2 Adaptive Workers — Ralph Loop
- 3.3 Scope Refinement Before the Ralph Loop
- 3.4 Iteration Boundaries
- 3.5 Reasoning Workers
- 3.6 Tool Proposal Validation

4. Tool Registry

- 4.1 Pre-Execution Hook — Snapshot Enforcement

5. Fine-Tuned Adapters

- 5.1 Proposal Capture Pipeline

6. Adversarial Review Tool

- 6.1 Call Modes
- 6.2 Invocation Interface
- 6.3 Harness Permission Scope

7. Block State — 2P-Set CRDT

8. Dashboard

- 8.1 Debug Mode (Phase 1 — Development)
- 8.2 Production Mode (Phase 2/3 — Status and Actionable)
 - Phase 2 — Status View
 - Phase 3 — Actionable View
- 8.3 Dashboard Architecture
- 8.4 Mode Switch

9. Event Taxonomy

10. Prototype Scope and Build Phasing

- 10.1 Target Task Classes
- 10.2 Build Phases
- 10.3 Open Questions

11. Key Design Decisions and Rationale

1. Architecture Overview

GRAIL (Goal-Regressive Adaptive Inference Layer) is a distributed autonomous agent architecture in which workers discover dependency structure by attempting the declared goal and regressing through failure. This prototype targets macOS system management on Apple M5 Max hardware, leveraging the MLX framework's native optimization for Apple Silicon unified memory.

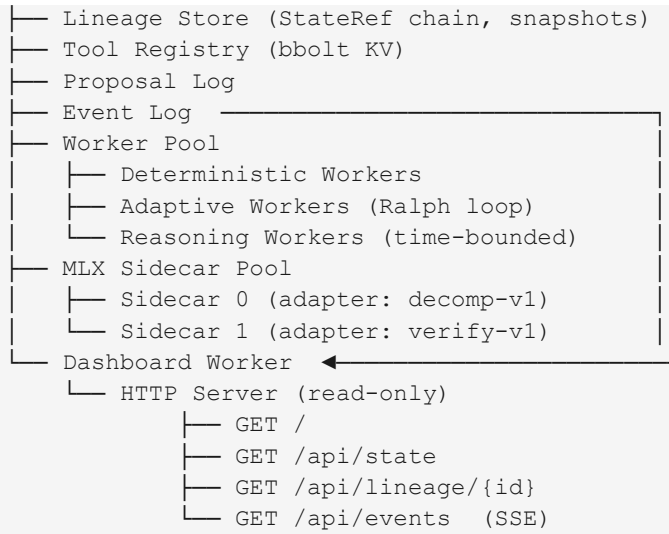
The prototype makes one deliberate structural compression relative to the full GRAIL whitepaper: the Process Manager and Scanner are fused into a single Go process on this single-node deployment. All data structures and interfaces are preserved so that the gossip transport can be added later for multi-node expansion without redesigning the state model.

1.1 Component Map

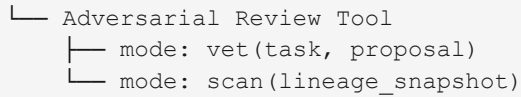
Component	Role
Process Manager / Scanner	Single Go process. Owns the task queue, block state CRDT, worker pool lifecycle, tool registry KV store, Ralph loop orchestration, and full lineage visibility (Scanner role).
Worker Pool	N goroutine-based stateless workers. Each accepts one task, executes it via deterministic tool or adaptive Ralph loop, returns a result, and resets.
MLX Sidecar Pool	One or more Python FastAPI processes serving a local inference API. Models loaded once into unified memory. LoRA adapters hot-swapped per request.
Tool Registry	KV store (bbolt prototype, etcd expansion path). Manager-owned writes. Workers read via in-memory cache with version staleness detection.
Block State	2P-Set CRDT. In-memory with WAL. Replicated via gossip in multi-node expansion.
Lineage Store	Immutable append-only StateRef chain per lineage. Snapshot records before every write/exec operation.
Proposal Log	Append-only capture of all tool proposals emitted by workers. Primary training data source for the proposal-v1 adapter.
Event Log	Append-only structured JSON event stream. Source of truth for the dashboard worker.
Dashboard Worker	Pure Go goroutine. Tails event log. Maintains in-memory dashboard state. No model involvement.
Dashboard HTTP Server	Pure Go. Read-only. Serves current state and SSE event stream to browser.
Adversarial Review Tool	External. pi-coding-agent harness wrapping MLX sidecar + fine-tuned adapter. Invoked by manager via subprocess call. Not part of the Go stack.

1.2 Stack Diagram

```
Process Manager / Scanner (Go)
├─ Task Queue (channel + WAL)
├─ Block State (2P-Set CRDT)
```



External (not Go):



2. Go ↔ MLX Bridge

MLX has no Go bindings. The bridge is a persistent Python FastAPI process — the MLX Sidecar — that Go workers call over localhost HTTP or Unix socket. This gives process isolation: an OOM or crash in the inference layer does not take down the Go coordinator.

The M5 Max's unified memory architecture is a genuine advantage here. Model weights loaded by MLX occupy shared DRAM accessible to both the GPU (via Metal) and the CPU (Go runtime) with no PCIe copy penalty. A 70B model at Q4 is approximately 35GB; two adapters can coexist in 128GB simultaneously, enabling adapter routing without eviction.

2.1 MLX Sidecar — Python

```
# grail_mlx_server.py
from fastapi import FastAPI
from mlx_lm import load, generate

app = FastAPI()
_models = {} # adapter_id -> (model, tokenizer)

def get_model(adapter_id: str):
    if adapter_id not in _models:
        model, tokenizer = load(
            BASE_MODEL_PATH,
            adapter_path=f"./adapters/{adapter_id}" if adapter_id else None
        )
        _models[adapter_id] = (model, tokenizer)
    return _models[adapter_id]

@app.post('/complete')
async def complete(req: CompleteRequest):
    model, tokenizer = get_model(req.adapter_id)
    response = generate(model, tokenizer,
        prompt=req.prompt, max_tokens=req.max_tokens, temp=req.temp)
    return {'text': response}

@app.get('/health')
async def health():
    return {'status': 'ok', 'loaded_adapters': list(_models.keys())}
```

2.2 Sidecar Pool — Go

```
type MLXSidecarPool struct {
    procs    []*MLXSidecarProcess
    clients  []*MLXClient
    mu       sync.Mutex
}

func (p *MLXSidecarPool) Start(n int) error {
    for i := 0; i < n; i++ {
```

```

    port := 8700 + i
    cmd := exec.Command('python3', 'grail_mlx_server.py',
        '--port', strconv.Itoa(port), '--host', '127.0.0.1')
    if err := cmd.Start(); err != nil { return err }
    p.procs = append(p.procs, &MLXSidecarProcess{cmd: cmd, port: port})
    p.clients = append(p.clients, newMLXClient(port))
}
return p.waitForHealthy(30 * time.Second)
}

// Route returns the client most likely to have adapter pre-loaded.
// Falls back to round-robin for unknown adapters.
func (p *MLXSidecarPool) Route(adapterID string) *MLXClient { ... }

```

3. Worker Execution Model

Workers operate in one of three modes determined by the task type and the Tool Registry entry for that task type. The mode is not chosen by the worker — it is a property of the task.

3.1 Deterministic Workers

Deterministic tasks have a registered tool in the Tool Registry. The worker's job is to select the correct tool and supply validated parameters. The tool layer constructs and executes the system call. The model never produces shell strings directly.

Representative macOS tools in scope:

Tool Name	Permissions	Description
brew_list	read	List installed packages and versions
brew_info	read, network	Fetch formula metadata from Homebrew API
brew_upgrade	exec	Upgrade a named package. Requires snapshot.
brew_uninstall	exec, CRITICAL	Uninstall a package. Requires adversarial review.
macos_snapshot	write	Create a timestamped system state record for this lineage.
softwareupdate_list	read	List available macOS updates.
softwareupdate_install	exec, CRITICAL	Install a named update. Requires snapshot + adversarial review.
cve_lookup	network read	Query NVD and OSV feeds. Scoped domains only.
system_profiler_read	read	Read specific system data types (no credentials).
firewall_rule_read	read	Read current firewall rules.
firewall_rule_modify	exec, CRITICAL	Modify firewall rules. Requires adversarial review.

3.2 Adaptive Workers — Ralph Loop

Adaptive tasks cannot be fully expressed as a single deterministic tool call. They require reasoning. The manager handles these using a Ralph-style loop: a fresh inference session is spawned per iteration with clean context. Persistence between iterations is externalized into the Lineage Store and a per-task progress log.

Ralph Loop — Core Principle

Each iteration spawns a fresh MLX inference session with clean context.
Memory persists via the Lineage StateRef chain and an append-only progress log.

The loop continues until the task resolves, the time budget is exhausted, or the manager detects a stall. Time-bounded, not depth-bounded. Patience is the design intent.

The Ralph seed assembled before the first iteration:

```
type RalphSeed struct {
    TaskDefinition      Task
    SystemSnapshot      StateRef    // current known system state
    RelatedWork         []StateRef // completed tasks in this lineage
    AvailableTools       []ToolEntry // scoped to this task's capability
    KnownConstraints     []string   // injected hard boundaries
    PriorAttempts        []IterationRecord // empty on first task
}
```

3.3 Scope Refinement Before the Ralph Loop

The worst input to a Ralph loop is an ambiguous or oversized task. Three refinement operations run in sequence before any Ralph loop starts:

1. Decomposition pass — a single scoped inference pass determines whether the task is atomic or needs splitting. If it needs splitting, subtasks are enqueued and the original task never enters a Ralph loop.
2. Context assembly — the manager assembles the fullest possible context snapshot: relevant StateRefs, applicable tools, known system state. This seeds the first iteration so it does not waste cycles orienting itself.
3. Constraint injection — the tool permission scope and ToolRegistry allowlist are written into the seed explicitly as hard boundaries. The model reads them before its first iteration, reducing proposals that fail structural validation and waste an iteration.

3.4 Iteration Boundaries

The manager evaluates four signals at each iteration boundary to determine whether the loop continues, pauses, or terminates:

Signal	Detection	Manager Action
Progress	Valid proposal passed structural validation	Execute, write StateRef, append learnings, spawn next iteration
Stall	Last 3 proposals similar (>85%) or same rejection reason	One intervention attempt (append stall notice), then invoke adversarial reviewer in Scan mode
Time Budget Warning	Wall time exceeds task allocation	Emit dashboard event, start grace period, then invoke adversarial reviewer in Scan mode

Signal	Detection	Manager Action
Risk Threshold	Proposal references a CRITICAL-rated tool	Pause loop, invoke adversarial reviewer in Vet mode before execution

3.5 Reasoning Workers

A third worker type handles tasks that are pure reasoning problems before they are action problems — for example, determining whether a specific CVE affects this machine's exact configuration. These workers run a tight inference loop time-bounded by the manager. Output is always a structured proposal or block declaration, never a direct system action.

3.6 Tool Proposal Validation

Every tool proposal produced by an adaptive or reasoning worker passes through two validation layers before execution:

- Structural validation (Go, fast, no model) — is the tool registered? Are parameters type-valid? Does the proposal exceed the worker's permission scope? Hard failures are returned to the loop with a rejection reason.
- Semantic validation (adversarial reviewer, invoked selectively) — is the proposed action actually correct for the stated goal? Does it introduce new risk? Invoked for CRITICAL-rated tools and on stall detection.

Proposal rejection loop depth:

- Structural failure — append rejection reason to progress log, spawn next iteration.
- Permission violation — Fatal immediately. No retry.
- Semantic failure — append finding as hard constraint, spawn next iteration.
- Repeated semantic failure — escalate to dashboard for human decision.

4. Tool Registry

The Tool Registry is a KV store owned exclusively by the Process Manager. Workers hold a read-only in-memory cache with version-based staleness detection. No worker ever blocks on a registry write.

```
type ToolEntry struct {
    Tool            Tool
    AddedAt         time.Time
    AddedBy         string          // 'human' | 'scanner' | 'proposal-analysis'
    SourceProposalIDs []string        // provenance: proposals that motivated this
    tool
    Status          ToolStatus      // active | deprecated | experimental
    RiskProfile     RiskLevel       // Low | Medium | High | Critical
    RequiresAdversarialReview bool
}

type ToolPermissions struct {
    CanRead      bool
    CanWrite     bool
    CanExec      bool
    ExecAllowlist []string // e.g. ['brew', 'softwareupdate']
    PathScope    []string // e.g. ['/opt/homebrew']
    NetworkScope []string // e.g. ['formulae.brew.sh', 'nvd.nist.gov']
}
```

The SourceProposalIDs field closes the loop between proposal capture and tool promotion. When the proposal-analysis adapter recommends promoting a recurring adaptive proposal to a deterministic tool, the manager writes it to the registry with full provenance. Every tool in the registry can be traced back to the proposals that motivated its creation.

The manager decides on tool promotion. The scanner can recommend promotions autonomously based on proposal analysis. Human approval is always required before a tool transitions from experimental to active status for CRITICAL-rated operations.

4.1 Pre-Execution Hook — Snapshot Enforcement

Every write or exec tool call requires a snapshot to exist for the current lineage before it fires. This is enforced at the tool layer, not left to the model or worker to decide.

```
func (t *Tool) ExecuteWithPolicy(
    ctx context.Context,
    params map[string]any,
    lineage *Lineage,
) (ToolResult, error) {
    if t.Permissions.CanWrite || t.Permissions.CanExec {
        _, err := lineage.RequireSnapshot(ctx)
        if err != nil {
            return ToolResult{}, fmt.Errorf("snapshot required: %w", err)
        }
    }
}
```

```
}  
result, err := t.Execute(ctx, params)  
// Write immutable StateRef regardless of success or failure  
lineage.AppendStateRef(ctx, t.Name, params, result, err)  
return result, err  
}
```

5. Fine-Tuned Adapters

Adapters are LoRA weights trained on top of a base model and loaded by the MLX sidecar at request time. Each adapter specializes a worker for a specific task geometry. Adapters are created as needed; the proposal log is the primary training data source.

Adapter ID	Task Types	Purpose
decomp-v1	decompose	Decompose high-level directives into atomic tasks. Trained on successful lineage dependency trees.
verify-v1	block_verify	Confirm or refute block claims. Trained on quorum confirmation/refutation outcomes.
summ-v1	state_summarize	Summarize accumulated lineage state into a compact StateRef for the next iteration.
assess-v1	terminal_assess	Assess whether the terminal task completion actually satisfies the original directive.
proposal-v1	all adaptive	Improve tool proposal quality and decisioning logic. Trained on proposal log with validation outcomes.
adversarial-v1	review (external)	Adversarial review. Trained to find false dependencies, premature block claims, and unsafe proposals. Lives in the external review tool, not the Go stack.

5.1 Proposal Capture Pipeline

Every tool proposal emitted by any worker — whether it passes validation, fails, or gets modified — is written to the Proposal Log with full context: task type, iteration number, prompt hash, proposal content, validation result, and execution outcome if applicable. This log is the training corpus for the proposal-v1 adapter and the signal source for tool promotion decisions.

```
type ProposalRecord struct {
    ID            string
    LineageID     string
    TaskID        string
    TaskType      string
    WorkerID      string
    Iteration     int           // Ralph loop iteration number
    PromptHash    string        // reproducibility
    Proposal      ToolCallProposal
    ValidationPass bool
    RejectionReason string
    ExecutionResult *ToolResult // nil if validation failed
    Timestamp     time.Time
}
```

6. Adversarial Review Tool

The Adversarial Review Tool is a standalone artifact — a pi-coding-agent harness wrapping an MLX sidecar loaded with the adversarial-v1 adapter. It is not part of the Go stack. It can be invoked by a human at a terminal, by a CI pipeline, or by the GRAIL Process Manager via a subprocess call. The Go stack treats it as an opaque external tool.

Design Principle

The Adversarial Review Tool is agnostic to its caller. It has a well-defined JSON interface on stdin/stdout. Nothing in its implementation knows or cares whether it was invoked by a human, the GRAIL manager, or another agent.

6.1 Call Modes

Mode	Input	Use Case
vet	task + specific proposal + system state	Review a single proposal before execution. Synchronous from manager perspective. Loop paused pending result.
scan	lineage snapshot + scan focus	Review a full lineage trajectory. Asynchronous. Finding influences next iteration boundary, not current one — unless severity is Critical.

6.2 Invocation Interface

```
// Manager invocation — subprocess call
type AdversarialReviewRequest struct {
    Mode          ReviewMode          // Vet | Scan
    Task          *Task               // Vet mode
    Proposal      *ToolCallProposal  // Vet mode
    Lineage       *LineageSnapshot   // Scan mode
    Focus         ScanFocus           // StallDetected | TimeBudgetWarning |
PostTerminal
    SystemState  StateRef
    TimeoutSecs int
}

type ReviewFinding struct {
    Mode          ReviewMode
    Severity      FindingSeverity // Info | Warning | Critical
    Summary       string
    Findings      []string
    Recommendation ReviewRecommendation // Continue | Revise | Escalate | Halt
}
```

6.3 Harness Permission Scope

The pi-coding-agent harness for the adversarial reviewer is intentionally constrained. It has read-only tools for inspecting system state and lineage records, and a single write tool: `emit_finding`. It cannot execute system commands, write to the filesystem, or modify lineage state. The role boundary is enforced structurally by the harness, not by prompt instruction.

Tool	Purpose
<code>read_file</code>	Read task definitions, StateRefs, and lineage records
<code>read_proposal_log</code>	Read proposal history for a task or lineage
<code>read_system_state</code>	Read current system state snapshot
<code>emit_finding</code>	The only write tool. Emits a structured ReviewFinding to stdout.

7. Block State — 2P-Set CRDT

Block state is maintained as a Two-Phase Set CRDT replicated across all nodes via gossip. On the single-node prototype, gossip is replaced by direct in-process channel communication. The data structure and interfaces are identical to what a distributed deployment uses.

```
type BlockSet struct {
    mu      sync.RWMutex
    Added   map[string]ActionableBlock // ClaimID -> block
    Removed map[string]BlockResolution // ClaimID -> resolution
}

// Merge is commutative, associative, idempotent.
// Removals always win — a resolved block cannot be re-activated.
func (bs *BlockSet) Merge(other BlockSet) {
    bs.mu.Lock()
    defer bs.mu.Unlock()
    for id, block := range other.Added {
        if _, exists := bs.Added[id]; !exists {
            bs.Added[id] = block
        }
    }
    for id, res := range other.Removed {
        bs.Removed[id] = res // removals always win
    }
}
```

On this single-node prototype, quorum degrades gracefully to a quorum of 1 per the whitepaper's `MinClusterSize` policy. Block claims are confirmed locally by the manager when only one worker is available to assess them. This is noted in the state record for future multi-node promotion.

8. Dashboard

The dashboard is pure Go — no model involvement, no inference, no MLX. It is an observation layer only. It cannot issue instructions to the manager. The HTTP layer is read-only in the prototype; intervention happens via a separate admin CLI.

The dashboard has two modes, switchable at startup and toggleable at runtime via a flag. Both modes share the same underlying event log and state model.

8.1 Debug Mode (Phase 1 — Development)

Debug mode is the initial capture mode used during development. The event emitter drops its severity filter — every trace-level event is written to the log. Tool parameter details are written unredacted. Proposal payloads are written in full.

The debug UI is a single HTML page with two panels:

- Live SSE event feed — raw events as they arrive, unformatted, with full payloads. The firehose. Invaluable for understanding exactly what the system is doing iteration by iteration.
- JSON state dump — the full in-memory DashboardState serialized and re-rendered on every event. No visual design. Maximum information.

Debug mode is intentionally low-effort to build. It confirms the event log data model is correct before any design work is invested in the production UI.

8.2 Production Mode (Phase 2/3 — Status and Actionable)

Production mode activates a severity filter (info and above) and sanitizes tool parameter payloads before logging (no credentials, no sensitive paths). The production UI layers over the same underlying state model.

Phase 2 — Status View

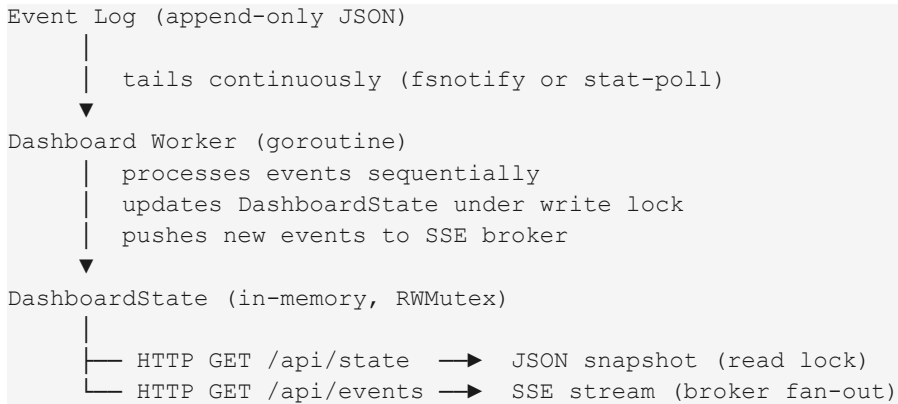
- Lineage cards showing goal text, status badge, task progress bar (complete / total), active block count, and Ralph loop iteration count with last-activity timestamp.
- Worker pool status: active workers, idle workers, MLX sidecar health indicators.
- System-level metrics: queue depth, events-per-minute, dropped events counter.
- Auto-updating via SSE — no polling interval to tune.

Phase 3 — Actionable View

- Escalation queue surfaced at top of page. Pending-review lineages displayed prominently with adversarial reviewer finding summarized inline.

- Per-lineage Ralph loop detail: iteration history with proposal hashes, validation outcomes, stall detection events, and time budget warnings.
- Snapshot timeline per lineage — visual record of when system state was captured and what triggered each snapshot.
- Admin CLI commands shown inline as copy-paste text for human intervention. Not clickable — HTTP layer remains read-only.

8.3 Dashboard Architecture



```

type DashboardState struct {
    mu          sync.RWMutex
    UpdatedAt   time.Time
    Workers     map[string]WorkerStatus
    MLXSidecars []SidecarStatus
    QueueDepth  int
    EventsDropped int
    Lineages    map[string]*LineageDashboard
}

type LineageDashboard struct {
    ID          string
    Goal        string
    Status      string // active|stalled|terminated|halted|pending_review
    TasksTotal  int
    TasksComplete int
    ActiveBlocks []BlockSummary
    RalphLoops  map[string]*RalphLoopDashboard
    ReviewFindings []ReviewFinding
    Snapshots    []SnapshotRecord
    RecentEvents []Event // ring buffer, last 50
}

type RalphLoopDashboard struct {
    TaskID      string
    Iteration   int
    Status      string
    WallDuration time.Duration
    Proposals   []ProposalSummary // ring buffer, last 20
  
```

```
    StallCount    int
}
```

8.4 Mode Switch

```
// Switchable at startup via flag, toggleable at runtime.
type DashboardMode int
const (
    DashboardDebug      DashboardMode = iota // full firehose, unredacted
    DashboardProduction // filtered, sanitized, designed
)

// Runtime toggle via admin endpoint (not the public HTTP handler)
// POST /admin/dashboard/mode { 'mode': 'debug' | 'production' }
// Admin endpoint binds to 127.0.0.1 only, separate port.
```

9. Event Taxonomy

All significant state transitions emit a structured event to the Event Log. The dashboard worker and any future observability tooling consume this stream. The taxonomy is the contract between the system internals and the observation layer.

Event Type	Severity	Description
lineage.created / terminated / stalled / halted	info / info / warn / error	Lineage lifecycle transitions
task.enqueued / dequeued / acked / requeued / fatal	debug / debug / debug / info / error	Task queue operations
block.claimed / confirmed / resolved	info	Block state transitions
ralph.iteration.start / end	debug	Ralph loop iteration boundaries
ralph.proposal.emitted / rejected / executed	debug / info / info	Proposal lifecycle
ralph.stall.detected	warn	Stall detection trigger
ralph.timebudget.warning	warn	Time budget threshold crossed
review.invoked / completed / escalated	info / info / warn	Adversarial review lifecycle
tool.executed / failed	info / error	Tool execution outcomes
snapshot.created	info	Lineage snapshot taken before mutation
worker.started / stopped	info	Worker pool lifecycle
mlx.sidecar.health	debug	Periodic sidecar health check result

10. Prototype Scope and Build Phasing

The prototype targets macOS system management as a concrete operational domain with genuine GRAIL characteristics: prerequisite sequences not known at invocation time, deterministic tool operations coexisting with adaptive reasoning tasks, and real consequences requiring adversarial review before execution.

10.1 Target Task Classes

Task Class	Execution Mode	Example Directives
Brew package management	Deterministic	"Update all outdated packages", "Remove unused dependencies"
macOS system updates	Deterministic + snapshot	"Apply all available security updates"
CVE assessment	Adaptive (Ralph loop)	"Determine if CVE-2025-XXXX affects this system"
Zero-day mitigation	Adaptive + adversarial review	"Identify and mitigate this zero-day across all affected components"
System hardening	Goal regression	"Harden all ingress and egress rules for this machine"

10.2 Build Phases

4. Foundation — Process Manager, Task Queue (channel + WAL), Block State (2P-Set CRDT), basic Worker goroutines with deterministic tool execution. No MLX yet. Validate the coordination model with mock workers.
5. MLX Integration — MLX Sidecar Pool, MLXClient, adapter routing. Validate inference round-trip latency and unified memory behavior with 2 adapters resident simultaneously.
6. Tool Registry — bbolt KV store, worker cache with version staleness, snapshot enforcement hook. Implement the macOS tool set defined in Section 3.1.
7. Ralph Loop — Adaptive worker type, RalphSeed assembly, scope refinement (decomposition pass, context assembly, constraint injection), iteration boundary logic, stall detection.
8. Dashboard Phase 1 — Event Log emitter, Dashboard Worker, HTTP handler with SSE. Debug mode only. Validate event taxonomy completeness against real execution.
9. Adversarial Review Tool — External pi-coding-agent harness, adversarial-v1 adapter, vet and scan modes. Integration with manager invocation.
10. Dashboard Phase 2/3 — Production mode UI layered over existing state model. Escalation queue. Ralph loop detail view. Mode switch.
11. Proposal Capture Pipeline — Proposal Log, analysis tooling, tool promotion workflow. Begin training data collection for proposal-v1 adapter.

10.3 Open Questions

- Tool promotion authority — the scanner can recommend tool promotions from proposal analysis. Should human approval always be required for CRITICAL-rated tool promotion, or can the manager promote Low/Medium risk tools autonomously after sufficient proposal evidence?
- Ralph loop time budget defaults — what is the right wall-time budget per adaptive task type? Depends on expected task complexity and MLX inference throughput at the chosen model size and quantization level. Needs empirical calibration.
- Reasoning worker convergence fallback — when a time-bounded reasoning worker does not fully converge, what is the output contract? Best-effort proposal with confidence flag, or explicit escalation to the Scanner?
- Multi-adapter concurrency — with 128GB unified memory and a 7B base model at Q4 (~4GB), many adapters can coexist. With a 70B model (~35GB), two instances fill the budget. Model size selection determines the concurrency ceiling and should be made early.

11. Key Design Decisions and Rationale

Decision	Rationale
Process Manager = Scanner (fused)	Single-node prototype has no gossip transport. Scanner's observability advantage is implicit when it runs inside the same process as queue and block state. Interfaces preserved for multi-node expansion.
Goroutine workers, not OS processes	Go scheduler on M-series hardware is well-suited to concurrent I/O-heavy workloads. OS processes add coordination overhead unnecessary on a single node.
HTTP sidecar for MLX, not cgo	Process isolation: MLX OOM or crash does not take down the Go coordinator. Serialization overhead is noise against LLM inference latency.
Ralph loop = fresh context per iteration	Prevents context pollution across long-running adaptive tasks. Externalized memory via StateRef chain is more reliable than accumulated in-context history.
Time-bounded Ralph loop, not depth-bounded	Patience is the design intent for non-deterministic tasks. Depth bounds punish legitimate long-running reasoning. Time bounds let the human decide when to interrupt.
Tool proposals, never shell strings	The model selects a registered tool and fills typed parameters. The tool layer constructs the system call. This distinction is the primary safety guarantee for system-mutating operations.
Adversarial reviewer is external	Keeps the Go stack clean. Makes the reviewer independently usable by humans and other agents. Enforces role boundary structurally via harness permissions.
Dashboard is read-only HTTP	Observation layer has no authority over the system. Intervention via separate admin CLI only. Prevents accidental mutation from the UI during development.
Proposal log as training corpus	Every proposal, pass or fail, is training data. The proposal-v1 adapter self-improves as the tool registry grows. Tool promotion decisions are evidence-based.